231501046
GANESH S AD23633

EXPERIMENT 6
Similar Product/Movie Retrieval using FAISS and Vector Embeddings

---

AIM:

To implement a program that retrieves similar movies based on high-dimensional vector embeddings using FAISS (Facebook AI Similarity Search) and Sentence Transformers on a movie dataset.

PROCEDURE:

1. Install required libraries: faiss-cpu, sentence-transformers, pandas.
2. Import pandas, numpy, faiss, and SentenceTransformer.
3. Create a sample movie dataset with 10 movies, each described by title and genre description.
4. Store the dataset in a pandas DataFrame with 'description' column.
5. Load the pre-trained embedding model: SentenceTransformer('all-MiniLM-L6-v2').
6. Encode all movie descriptions into 384-dimensional vectors.
7. Build FAISS IndexFlatL2 with the embedding dimension.
8. Add all embedding vectors to the FAISS index.
9. Define recommend(movie_query, k=3):
   a. Encode the query string into a vector.
   b. Search the FAISS index for k nearest neighbors.
   c. Return and print the top-k most similar movie descriptions.
10. Test the recommendation system with queries: 'space science movie', 'romantic love story', 'action revenge movie'.

CODE:

```
!pip install faiss-cpu sentence-transformers pandas --quiet

import pandas as pd, numpy as np, faiss
from sentence_transformers import SentenceTransformer

movies = [
    'The Matrix - Sci-fi action about virtual reality',
    'Titanic - Romantic drama on a sinking ship',
    'Inception - Mind-bending dream thriller',
    'Avengers - Superhero action movie',
    'Interstellar - Space exploration and time dilation',
    'The Notebook - Emotional romantic story',
    'John Wick - Action-packed revenge story',
    'Gravity - Survival in space',
    'Iron Man - Tech superhero origin story',
    'The Conjuring - Horror supernatural investigation'
]

df = pd.DataFrame(movies, columns=['description'])
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(df['description'].tolist())

dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(embeddings))
print('FAISS index built with', index.ntotal, 'movies')

def recommend(movie_query, k=3):
    query_vector = model.encode([movie_query])
```

```
    distances, indices = index.search(np.array(query_vector), k)
    print(f'\nQuery: {movie_query}')
    print('Recommended Movies:')
    for i in indices[0]:
        print('-', df.iloc[i]['description'])

recommend('space science movie')
recommend('romantic love story')
recommend('action revenge movie')
```

OUTPUT:

FAISS index built with 10 movies

🔍 Query: space science movie

🎬 Recommended Movies:

- Avengers - Superhero action movie
- Gravity - Survival in space
- Interstellar - Space exploration and time dilation

🔍 Query: romantic love story

🎬 Recommended Movies:

- The Notebook - Emotional romantic story
- Titanic - Romantic drama on a sinking ship
- Inception - Mind-bending dream thriller

🔍 Query: action revenge movie

🎬 Recommended Movies:

- Avengers - Superhero action movie
- John Wick - Action-packed revenge story
- The Matrix - Sci-fi action about virtual reality

RESULT:

The FAISS-based vector similarity search system was successfully implemented. The FAISS index was built with 10 movies. For 'space science movie', it recommended Avengers, Gravity, and Interstellar. For 'romantic love story', it recommended The Notebook, Titanic, and Inception. For 'action revenge movie', it recommended Avengers, John Wick, and The Matrix. The results confirm that semantic similarity search using dense embeddings provides meaningful and contextually relevant recommendations.